

PROJECT REPORT

Digital Marketing Service Website

Submitted to:

Mam Salma Iqbal

Submitted by:

Haider Jalil Malik

AIR UNIVERSITY ISLAMABAD

National Cyber Security Academy
(NCSA)

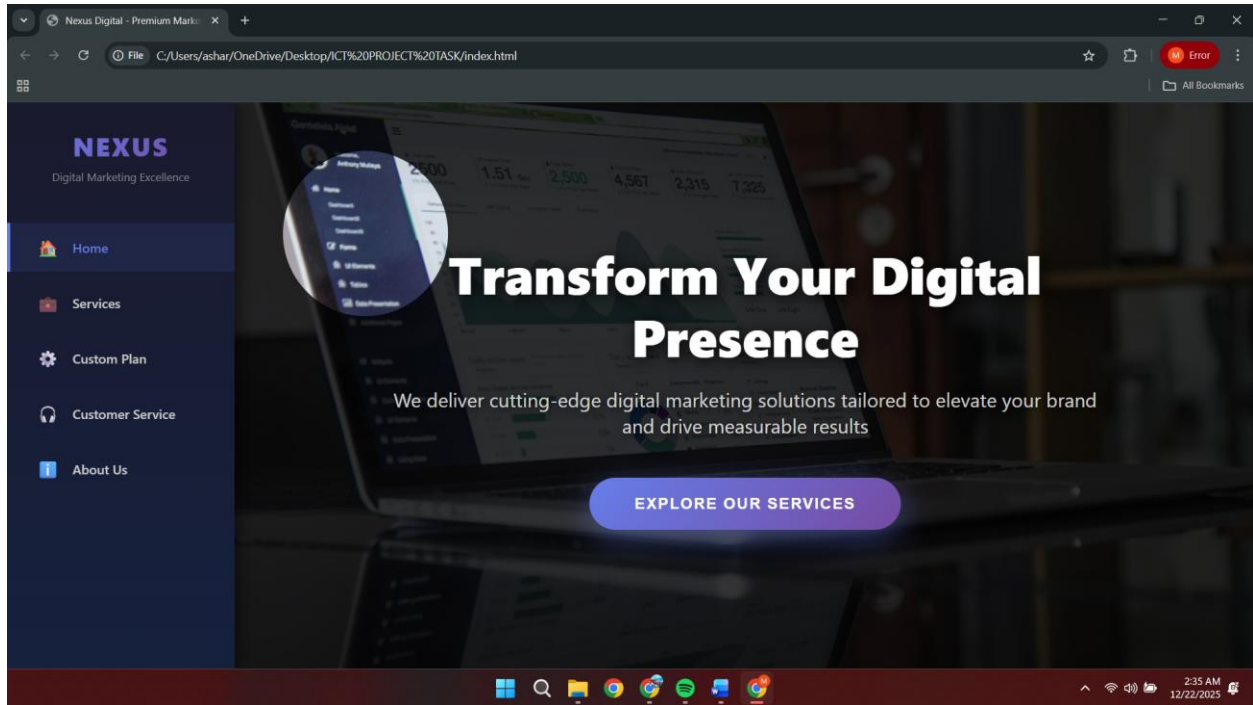
Table of Contents

1. Introduction.....	3
2. Problem Statement.....	3
3. Objectives of the Project.....	3
4. Scope of the Project.....	4
4.1 Included Scope	4
4.2 Excluded Scope	5
5. System Overview.....	5
6. Target Users.....	6
7. Development Methodology.....	6
8. Tools and Technologies Used	7
9. Key Features of the System	8
10. User Interface and Design Analysis (CSS)	9
10.1 Design Principles Used	10
10.2 Layout Structure	10
10.3 Color Scheme and Typography	11
10.4 Animations and Transitions.....	12
10.5 Responsive Design	12
10.6 Forms Styling and User Interaction	13
10.7 Accessibility Considerations	13
11. HTML Structure and Page Organization	13
11.1 Overall HTML Layout	14
11.2 Navigation Structure.....	15
11.3 Home Page Structure	15
11.4 Services Page Structure	16
11.5 Custom Plan Form Structure	16
11.6 Customer Service and Complaint Section.....	17
11.7 About Us and Testimonials Section	17
11.8 Semantic Elements Used	18
12. JavaScript Functionality and Logic	18
12.1 Page Navigation Logic.....	19

12.2 Event Handling	19
12.3 DOM Manipulation	20
12.4 Interactive Home Page Reveal Effect	20
12.5 Service Package Selection Logic	20
12.6 Form Handling and Validation	21
12.7 User Feedback and Alerts	21
12.8 Performance and User Experience Enhancements	22
12.9 Error Prevention and Reliability	22
13. Testing and Validation	23
13.1 Frontend Testing	23
13.2 Backend Testing.....	23
14. Limitations of the System	23
15. Future Enhancements.....	24
16. Conclusion	24

1. Introduction

In today's world, you basically don't exist if your business isn't online. Our project, Nexus Digital – Premium Marketing Solutions, is a site we built for a digital marketing agency using HTML, CSS, and JS. We wanted it to look super clean and professional while showing off their services, prices, and ways for people to get in touch. We went with a single-page vibe, so everything flows together without having to wait for pages to reload. It just makes the whole thing feel way faster and smoother to use.



2. Problem Statement

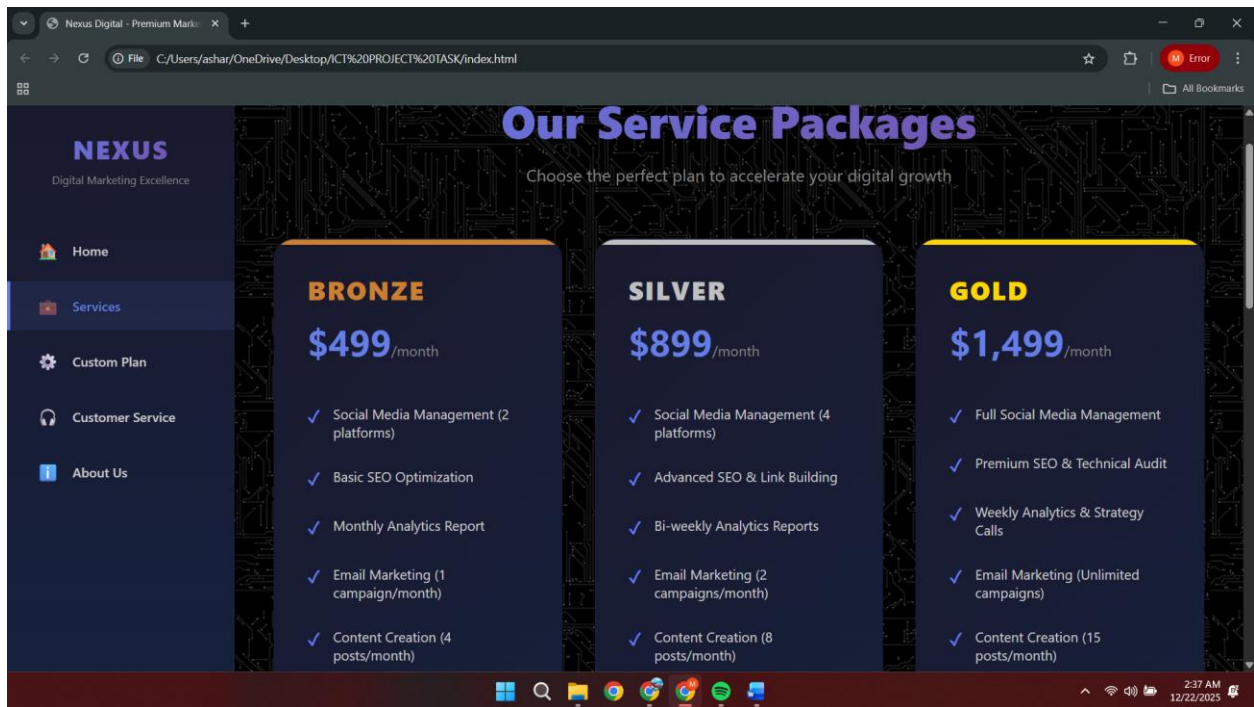
A lot of businesses out there really struggle because their websites look messy or don't clearly show what they do and how much they charge. When a site is hard to use, people usually just leave and don't trust the brand. Our project fixes this by creating a clean, interactive, and responsive site for a marketing agency. It's basically a real-world example of how a professional platform should work, while also letting us show off what we can do with front-end development.

3. Objectives of the Project

For the Nexus Digital project, these were the main things we wanted to get done:

- Build a website that looks professional and actually feels interactive.

- Use JavaScript so the navigation feels smooth and dynamic.
- Lay out different service packages with clear pricing so it's easy to understand.
- Make the whole experience better by adding animations and working forms.
- Make sure the UI is fully responsive, so it looks good on any screen.



4. Scope of the Project

4.1 Included Scope

For this project, we designed and built a full digital marketing platform that handles both the frontend and the backend. Our goal was to create a site that feels like a real-world application, so we made sure it could actually process and store user information. Here is a breakdown of what we built:

- **Responsive Frontend:** We created a single-page website using HTML, CSS, and JavaScript. We made sure it was interactive and responsive so that it looks and works great on both phones and computers.
- **Backend Server:** To power the site, we developed a server using Node.js and Express.js. This acts as the "brain" of our platform and manages all the requests.
- **Form Submissions & APIs:** We set up an API system so users can submit their own custom marketing plans or reach out to customer support.

- **Data Storage:** To make the data permanent, we linked the forms to Excel files. This way, any info a user submits is saved in a spreadsheet rather than just disappearing.
- **Dependency Management:** We implemented a package.json configuration to strictly define the project's version and dependencies. This ensures that the server environment is consistent and easy to set up using standard NPM commands.
- **Automatic ID Generation:** Every time a user submits a plan or a complaint, the system automatically creates a unique ID for that request so it's easy to track.
- **Input Validation:** We also added basic validation to our forms. This ensures that users fill out all the required fields and don't send any incomplete info.

4.2 Excluded Scope

The following features are not included in the current implementation:

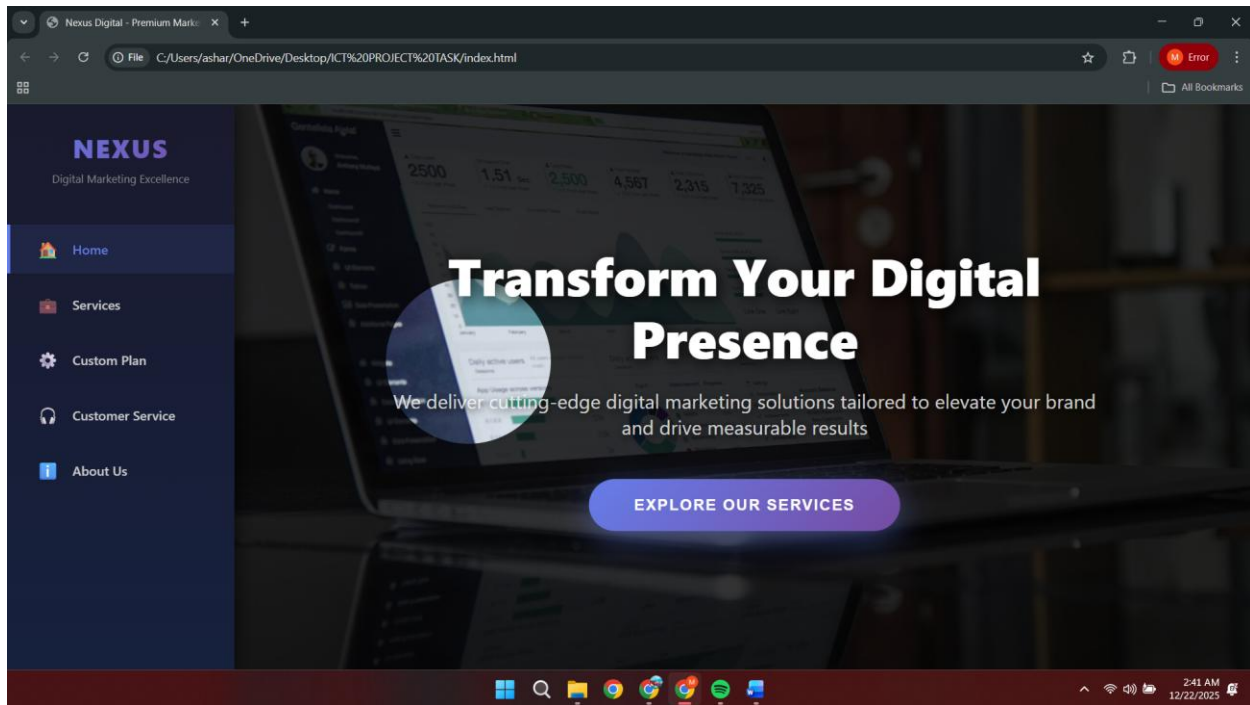
- User authentication and login system
- Online payment processing
- Advanced database systems such as MySQL or MongoDB
- Admin dashboard for managing submitted records
- Deployment on a live production server

5. System Overview

Our project, Nexus Digital – Premium Marketing Solutions, is a web app that we split into two main parts: the frontend and the backend.

For the frontend, we used HTML, CSS, and JavaScript. We focused on making the interface interactive and responsive so it feels smooth for the user. On the other side, the backend is powered by Node.js and Express.js. This is the part of the system that actually does the heavy lifting, like processing forms and making sure all the data gets saved correctly.

The two sides talk to each other through API requests. Whenever someone fills out a form on the site, that data is sent to our backend. From there, our code checks the input for errors, assigns it a unique ID, and saves it into an Excel file. By setting it up this way, we made sure the data is stored permanently and handled reliably.



6. Target Users

We basically built this site for a few different types of people:

- Business owners who need to get their brand out there with digital marketing.
- Startups that are trying to compare different service packages to see what fits their budget.
- Future clients who want to reach out and get a marketing plan made just for them.

We made sure the whole site is super easy to use, so it doesn't matter if you're a tech expert or have no clue how websites work anyone can get around it.

7. Development Methodology

We built this project using a modular process to keep things organized. Here is how we broke it down:

- Figuring out the requirements to see what the site actually needed.
- Planning the UI and layout so we had a solid blueprint before coding.
- Building the HTML structure to get the bones of the site in place.
- Adding CSS for all the styling and smooth animations.

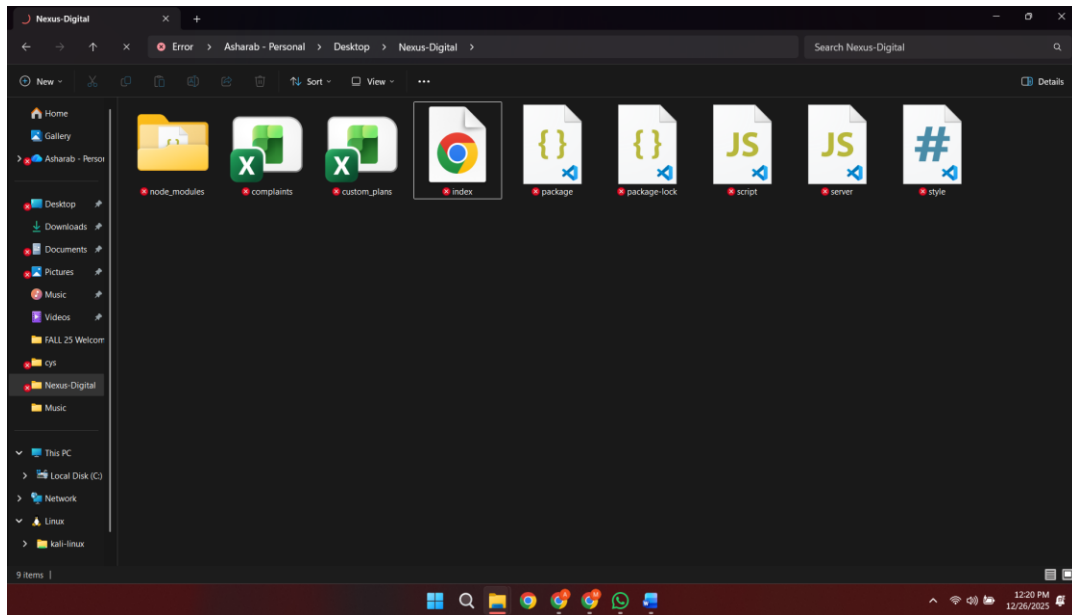
- Writing the JavaScript to handle the logic and make everything interactive.
- Testing and fixing bugs to make sure everything worked perfectly.

Doing it this way makes the code way easier to manage and lets us add new features later on without any headaches.

8. Tools and Technologies Used

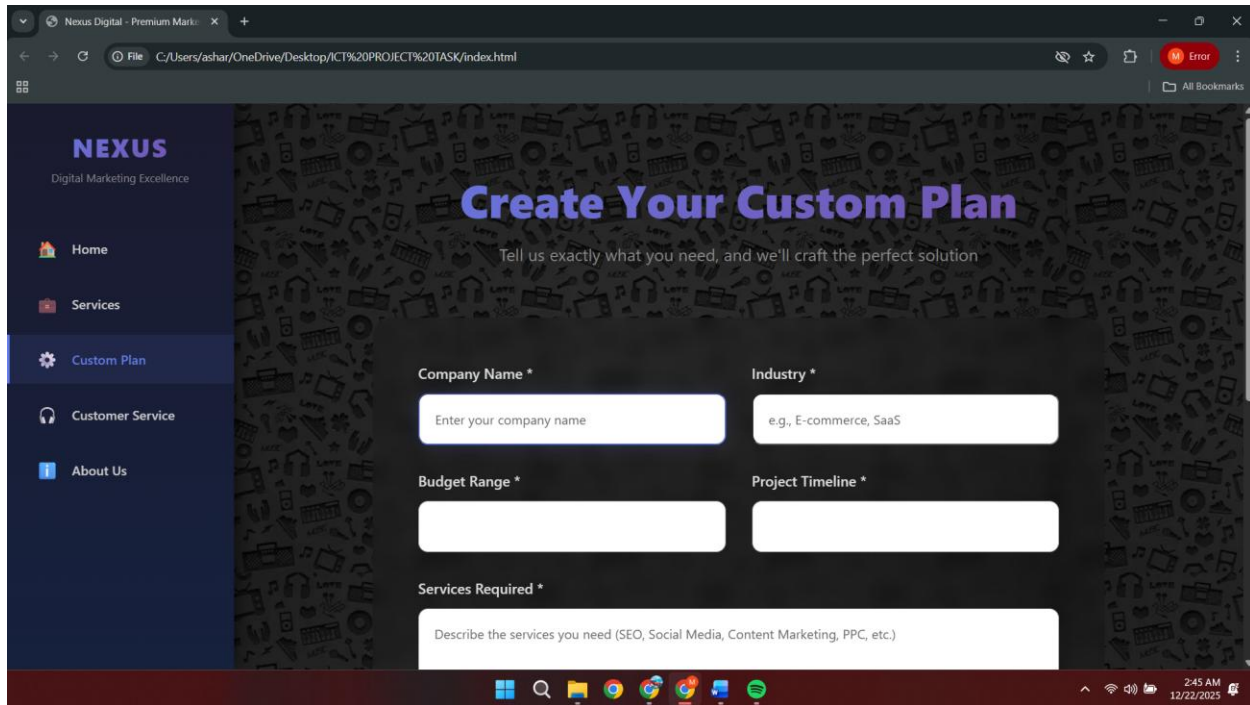
To build this project, we used a variety of tools that helped us handle everything from the design to the server logic. Here is what we used:

- Frontend Basics (HTML, CSS, and JavaScript): We used HTML to structure the pages, CSS to handle the layout and make sure it looked good on different screens, and JavaScript to make the site interactive.
- JSON (JavaScript Object Notation): We used JSON as the standard format for sending data between our frontend and backend. When a user submits a form, the data is converted into a JSON string before being sent to the server, which ensures the information is structured and easy for the backend to process.
- Node.js & Express.js: These were the main tools for our backend. Node.js let us run the server, while Express.js made it much easier to build and manage our APIs.
- NPM & package.json: We used NPM (Node Package Manager) to install all the libraries our project depends on. The package.json file acts as the configuration center for our backend; it tracks all our installed modules (like Express and XLSX) and holds the executable scripts that start our server.
- CORS: We included this to make sure our frontend, and backend could communicate with each other securely without any errors.
- XLSX Library: Since we needed a way to save data, we used this library to write the form results into Excel files.
- Nodemon: This was a huge help during development because it automatically restarted our server whenever we made changes to the code.
- Web Browser: We used the browser (like Chrome or Edge) constantly to test our features and use the developer tools to debug any issues we ran into.



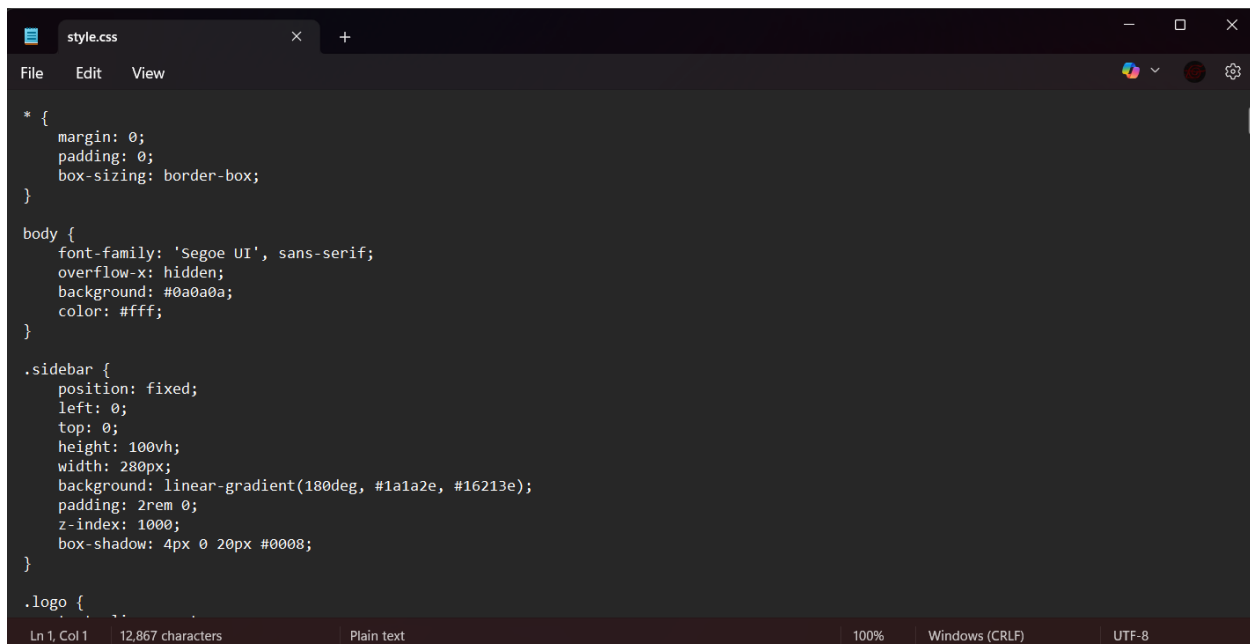
9. Key Features of the System

- Dynamic switching – You can hop between different sections instantly without the page ever having to reload.
- Interactive service cards – We made the package options look really good and react when you interact with them.
- Custom plan form – A dedicated spot where users can hit us up for a plan that fits their specific needs.
- Support module – A built-in area for customer service and for people to drop any complaints or feedback.
- Fully responsive – We made sure the layout shifts perfectly so it looks just as good on a phone as it does on a laptop.



10. User Interface and Design Analysis (CSS)

Everything you see on the Nexus Digital site and how it feels to use mostly comes down to the CSS. We used it to handle the layout, the colors, and the fonts, plus all the animations and making sure it looks right on different screens. Our main goal was to keep the design modern and clean so it's easy for anyone to use without getting confused.



10.1 Design Principles Used

When we were working on the CSS for Nexus Digital, we stuck to a few main goals to make sure it looked right:

- Consistency: We used the same colors, fonts, and styles across the whole site so nothing feels out of place.
- Visual Hierarchy: We made sure the big stuff like headings and buttons really stands out by playing around with their size, color, and spacing.
- Simplicity: We kept things chill and avoided clutter so the content is actually easy to read.
- User Engagement: We added hover effects and smooth transitions to make the site feel more "alive" and fun to click through.

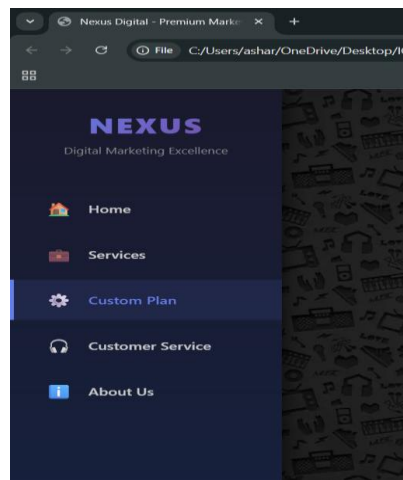
Basically, keeping things simple and organized makes the site way less confusing and just better to use.

10.2 Layout Structure

The site uses a sidebar layout paired with a main content area, which gives it that professional, dashboard-style look while making it super easy to navigate.

- The sidebar stays locked on the left side of the screen so the navigation links are always there when you need them.
- The main area is where all the action happens, switching up the content based on what the user clicks.

To keep the layout from breaking, we used CSS properties like position: fixed, margin-left, and min-height so everything stays exactly where it's supposed to be, no matter what screen size you're on.



10.3 Color Scheme and Typography

Color Scheme

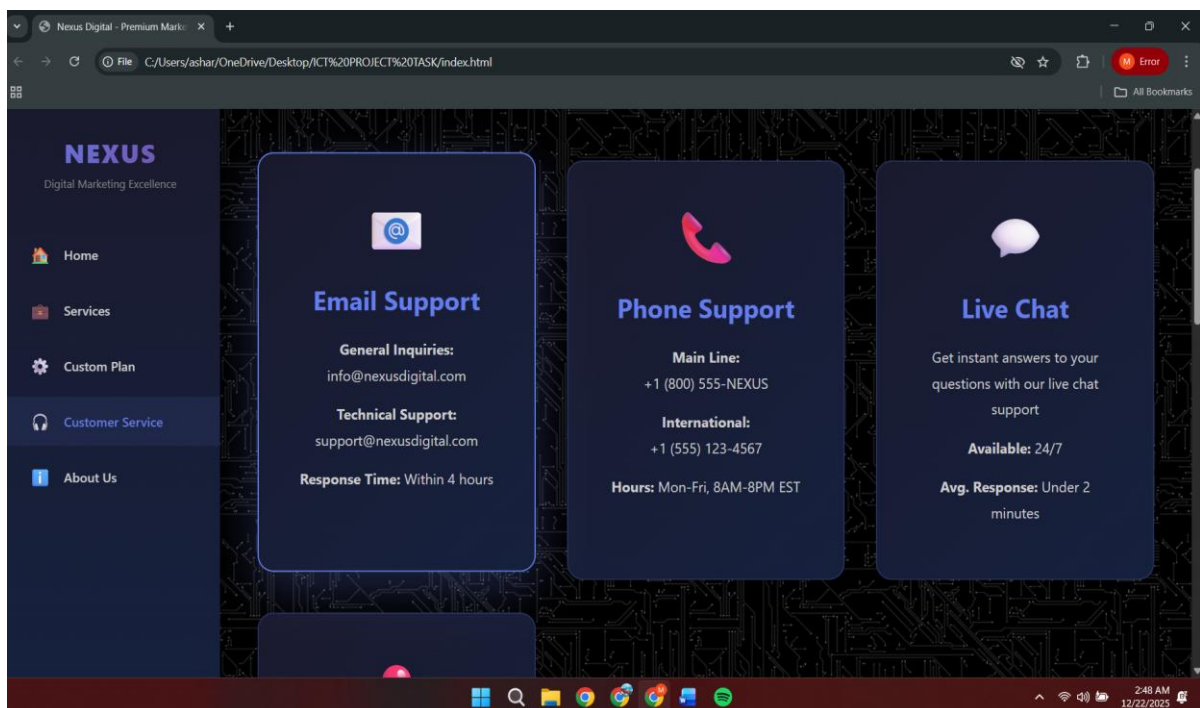
We went with a dark theme for the site and mixed in some gradient highlights to make it pop. Here is why:

- **Dark backgrounds:** These make it a lot easier on the eyes, especially if you're looking at the screen for a while.
- **Gradient accents:** We used these to help with branding and to make the important stuff stand out.
- **Consistent highlight colors:** All the buttons and links use the same colors so the whole thing feels tied together.

This combo really levels up the look and gives the site that high-end, professional vibe.

Typography

- We used a clean, sans-serif font because it's super easy to read on a screen.
- The headings are big and have gradient effects to catch your eye.
- For the paragraphs, we kept the contrast at a good level so it's comfortable to read through the info.

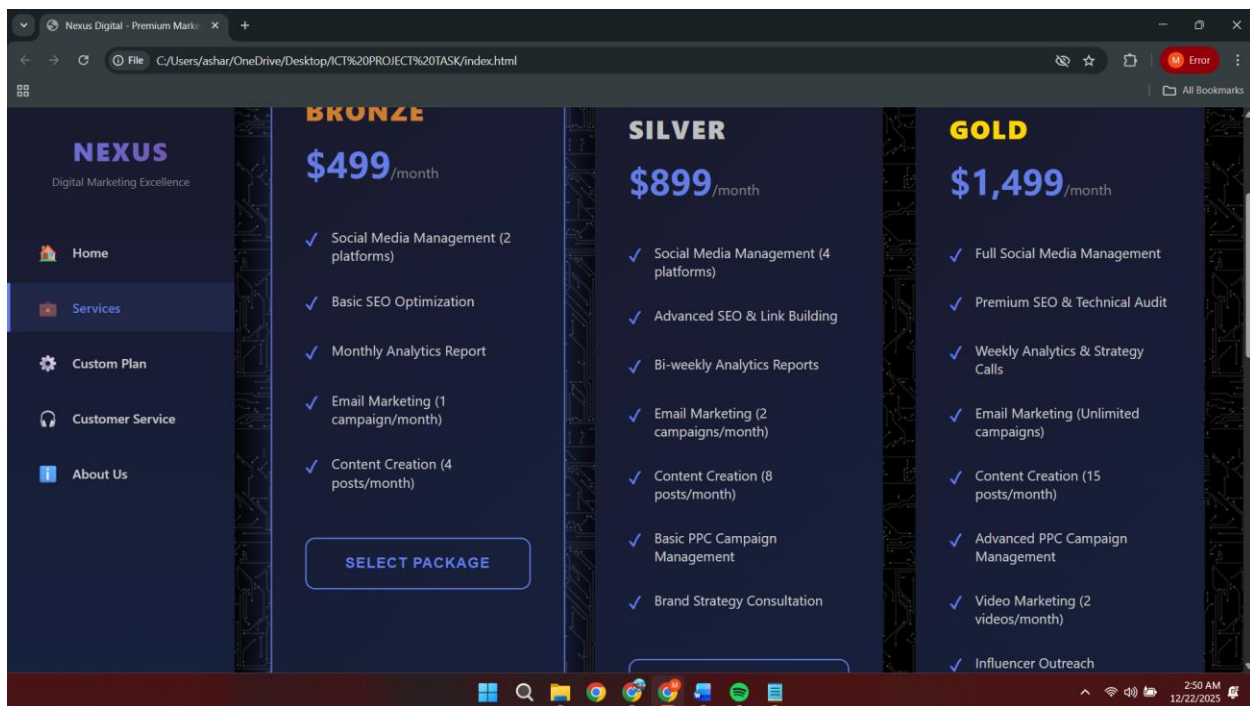


10.4 Animations and Transitions

We used a lot of CSS animations and transitions to make the site feel more polished without going overboard. Here's some of the stuff we added:

- Fade-in effects so it looks smooth when you're switching between different sections.
- Hover animations on buttons and cards so you know exactly what you're about to click.
- Smooth transitions for the navigation links to make the sidebar feel more interactive.

To get these working, we used things like, `@keyframes`, `transition`, and `transform`. These little details are great because they give the user instant feedback and just make the whole site feel way more responsive and high-quality.



10.5 Responsive Design

We made sure the site is fully responsive so it looks good whether you're on a laptop, a tablet, or just scrolling on your phone. Here is how we handled it:

- Media queries to tweak the layout whenever the screen gets smaller.
- Flexible grid layouts using CSS Grid so everything snaps into place naturally.
- Scalable fonts and containers that resize themselves so nothing looks tiny or huge.

Basically, as the screen shrinks, the layout automatically shifts things around so the site stays easy to use and doesn't get all cluttered.

10.6 Forms Styling and User Interaction

We used CSS to make the forms for custom plan requests and complaints actually look good and easy to fill out. Here is what we focused on:

- Clearly labeled inputs so you aren't guessing what you're supposed to type.
- Focus effects that highlight the box you're currently typing in, which helps guide you through the form.
- Styled buttons that match the rest of the site's theme so everything stays consistent.

By styling the forms this way, they're way less intimidating to use, which definitely helps get more people to actually interact with them.

10.7 Accessibility Considerations

We tried to keep basic accessibility in mind so everyone can actually use the site. Here's what we did:

- Good color contrast: We made sure the text stands out from the background so it's not hard to see.
- Clear buttons and links: It's super obvious when you're hovering over or clicking something.
- Readable fonts: We didn't use tiny text that makes you squint.

We didn't go crazy with advanced accessibility tools, but the design is definitely solid enough for most people to get around without any issues.

11. HTML Structure and Page Organization

HTML is basically the skeleton of the Nexus Digital site. We used it to map out all the different sections and create containers for all the stuff CSS and JavaScript eventually fill in. Since we went with a single-page application (SPA) vibe, we put the entire structure into one file and just swap out what you see on the screen dynamically

```

index.html
File Edit View
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<title>Nexus Digital - Premium Marketing Solutions</title>
<link rel="stylesheet" href="style.css">
</head>
<body>
<div class="sidebar">
<div class="logo">
<h1>NEXUS</h1>
<p>Digital Marketing Excellence</p>
</div>
<div class="nav-links">
<li><a href="#" class="nav-link active" data-page="home"><span>🏠</span>Home</a></li>
<li><a href="#" class="nav-link" data-page="services"><span>🔧</span>Services</a></li>
<li><a href="#" class="nav-link" data-page="custom-plan"><span>⚙️</span>Custom Plan</a></li>
<li><a href="#" class="nav-link" data-page="customer-service"><span>👤</span>Customer Service</a></li>
<li><a href="#" class="nav-link" data-page="about"><span>📄</span>About Us</a></li>
</div>
<div class="main-content">
<div id="home" class="page active">
<div class="reveal-container">
<div class="base-layer"></div>
<div class="top-layer" id="topLayer"></div>
<div class="home-content">
<h1>Transform Your Digital Presence</h1>
<p>We deliver cutting-edge digital marketing solutions tailored to elevate your brand and drive measurable results</p>
<button class="cta-btn" onclick="navigateToPage('services')">Explore Our Services</button>
</div>
</div>
<div id="services" class="page">
<div class="page-header">
<h2>Our Service Packages</h2>
<p>Choose the perfect plan to accelerate your digital growth</p>
</div>
</div>
</div>
</body>
</html>
Ln 1, Col 1 12,976 characters Plain text 100% Windows (CRLF) UTF-8

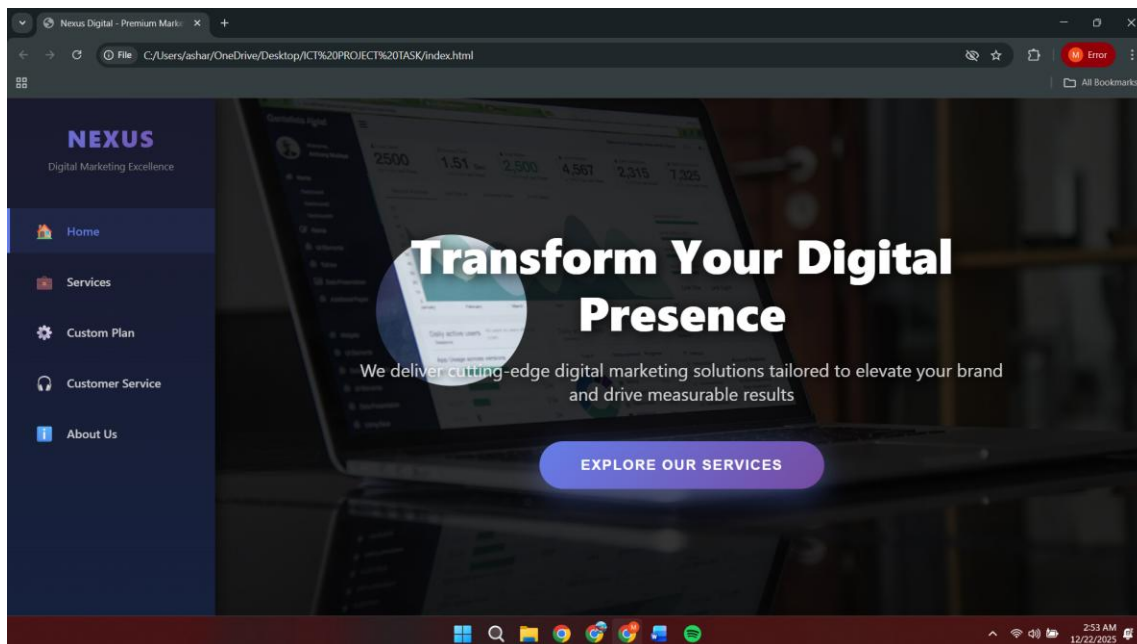
```

11.1 Overall HTML Layout

We organized the HTML layout in a really straightforward way by splitting it into two main parts:

- The Sidebar: This is where the logo lives and all the navigation links are kept.
- The Main Content: This is the big area where the actual info for each page pops up.

Dividing it like this keeps the navigation separate from the content, which makes the site way easier to use and much simpler to manage if we need to change things later.



11.2 Navigation Structure

For the navigation, we basically set up an unordered list inside the sidebar. Each link is tied to a specific section of the site using custom attributes so the system knows exactly what to show.

Here's the main deal with it:

- Mapped links: Every click takes you straight to the right section.
- Active highlights: It's easy to see which page you're currently on because the link lights up.
- Smooth transitions: It doesn't feel jumpy when you move around.

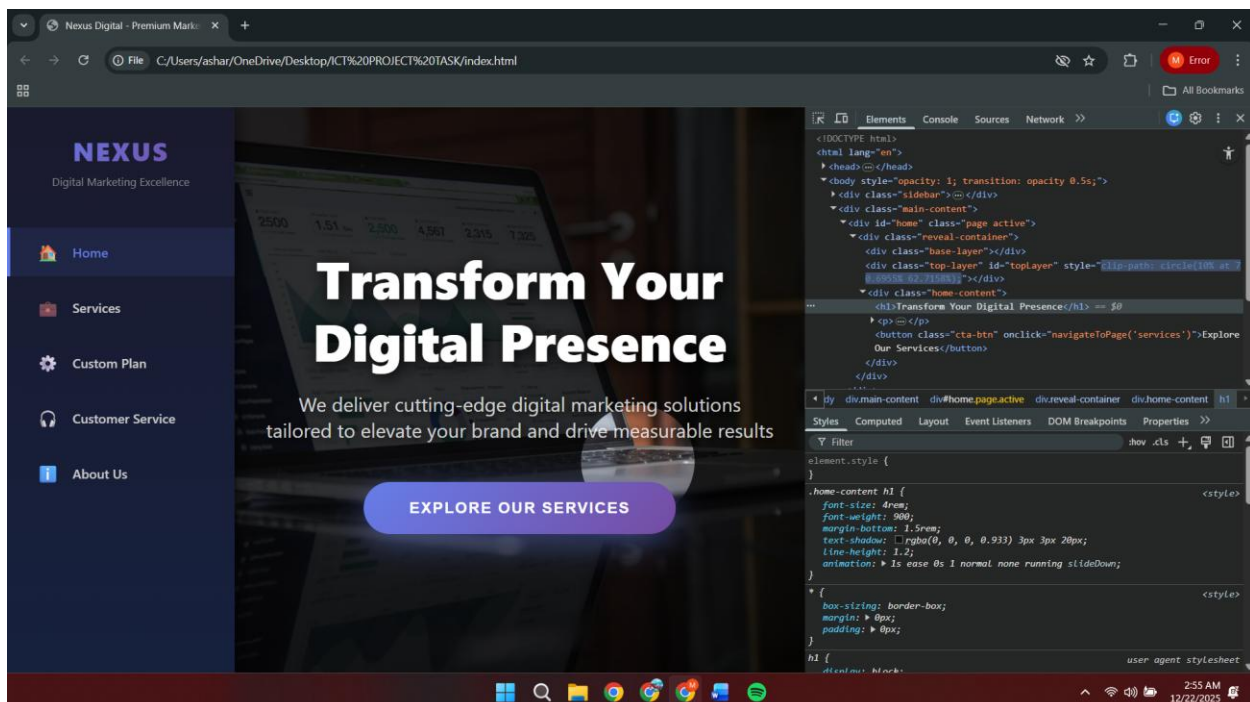
Setting it up this way makes navigation super-fast since the page never has to fully reload.

11.3 Home Page Structure

The home section is basically the first thing people see when they land on the site. We kept it simple but effective by including:

- A hero heading to grab everyone's attention right away.
- A descriptive paragraph that explains exactly what we're about.
- A call-to-action button to get people moving to the next step.

We also added some layered elements in the background so we could pull off some cool interactive visual effects using CSS and JavaScript.

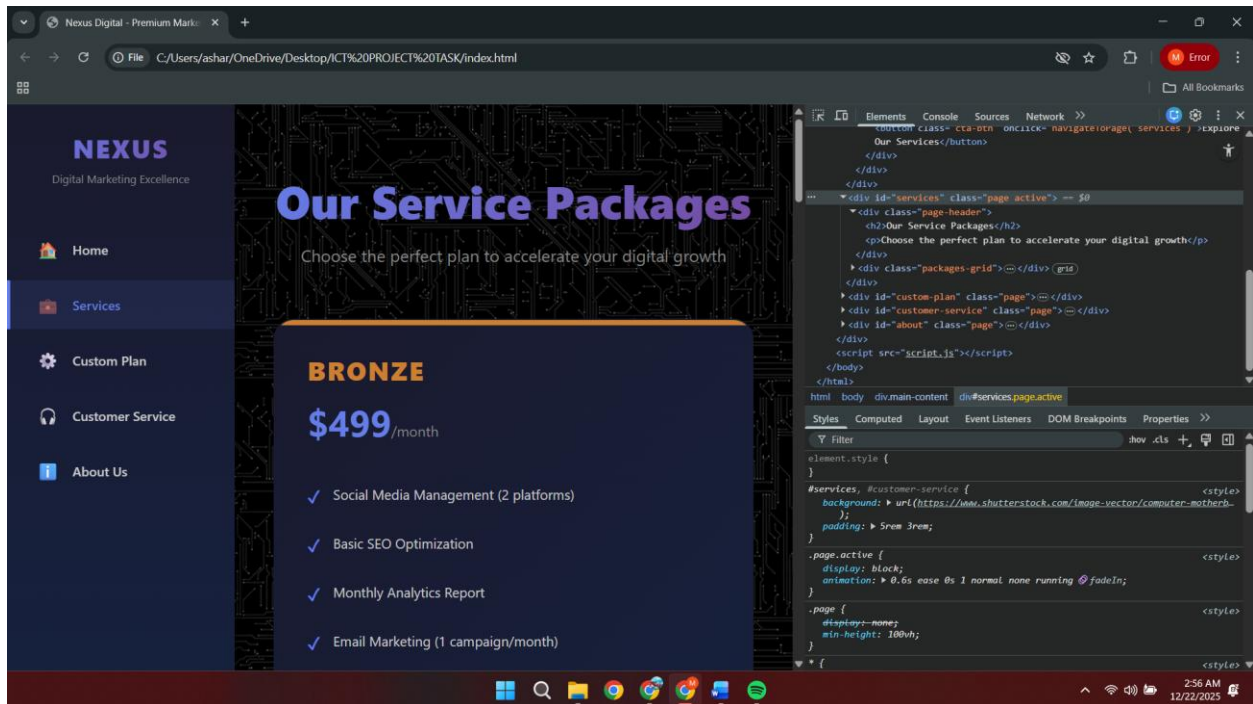


11.4 Services Page Structure

The services section is where we show off the different digital marketing packages using organized content blocks. Each service card has the same basic setup:

- The package name so you know what you're looking at.
- Pricing info so there are no surprises.
- A feature list that breaks down exactly what you get.
- A selection button to make it easy to pick one.

Keeping all the service cards structured the same way makes everything a lot easier to read and way simpler for us to style with CSS.

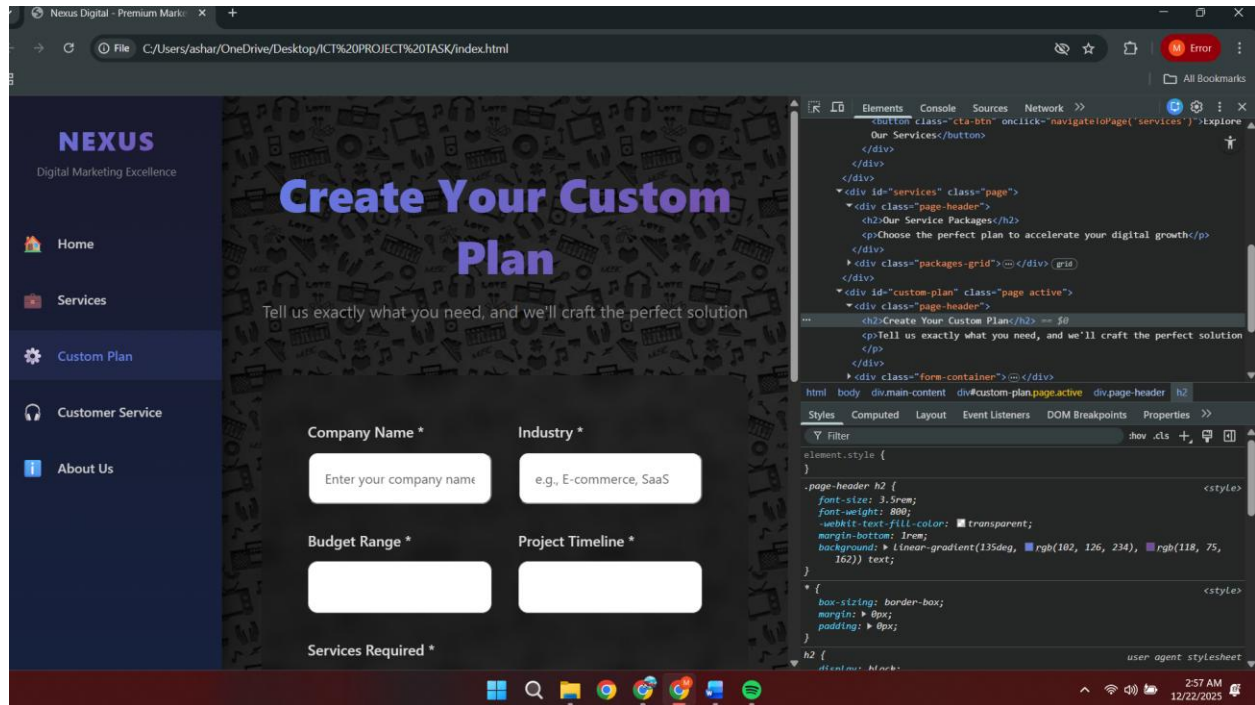


11.5 Custom Plan Form Structure

The custom plan section has a form where users can send over their specific business needs. We built it with a mix of different fields to gather all the right info:

- Text input fields for basic details.
- Dropdown menus to make selecting options quick and easy.
- Text areas so people can actually write out the specifics of what they're looking for.
- A submit button to send it all through.

We also used HTML validation attributes, like required, to make sure nobody accidentally skips an important box before hitting send.



11.6 Customer Service and Complaint Section

This part of the site is where we put all the contact info and a spot to submit complaints. It includes:

- Email and phone details so people can get ahold of us easily.
- Office location info to show where we're actually based.
- A complaint form so users have a direct way to give us feedback or vent.

Setting it up like this is a big deal for transparency it shows we're real people and helps build a lot of trust with anyone visiting the site.

11.7 About Us and Testimonials Section

The About Us section is basically where we tell the story of the brand. We used a mix of different elements to keep it organized:

- A company description to give everyone the lowdown on who we are.
- Business stats to show off some of the numbers and wins we've had.
- Customer testimonials so people can see what others actually think of us.

We used standard HTML stuff like headings, paragraphs, and cards to lay everything out so it's not just a giant wall of text.

11.8 Semantic Elements Used

We used semantic HTML tags to build the site, which basically means the code itself tells you what each part is for. Some of the main ones we used include:

- `<header>` for the top-level stuff.
- `<section>` to break the page into logical parts.
- `<form>` for all the user input areas.
- `<ul>` and `<li>` to keep our lists and navigation menus organized.

Using these specific tags is a huge plus because it makes the site way easier for screen readers to navigate, keeps the code clean for us to read, and even helps out with SEO so the site is easier to find.

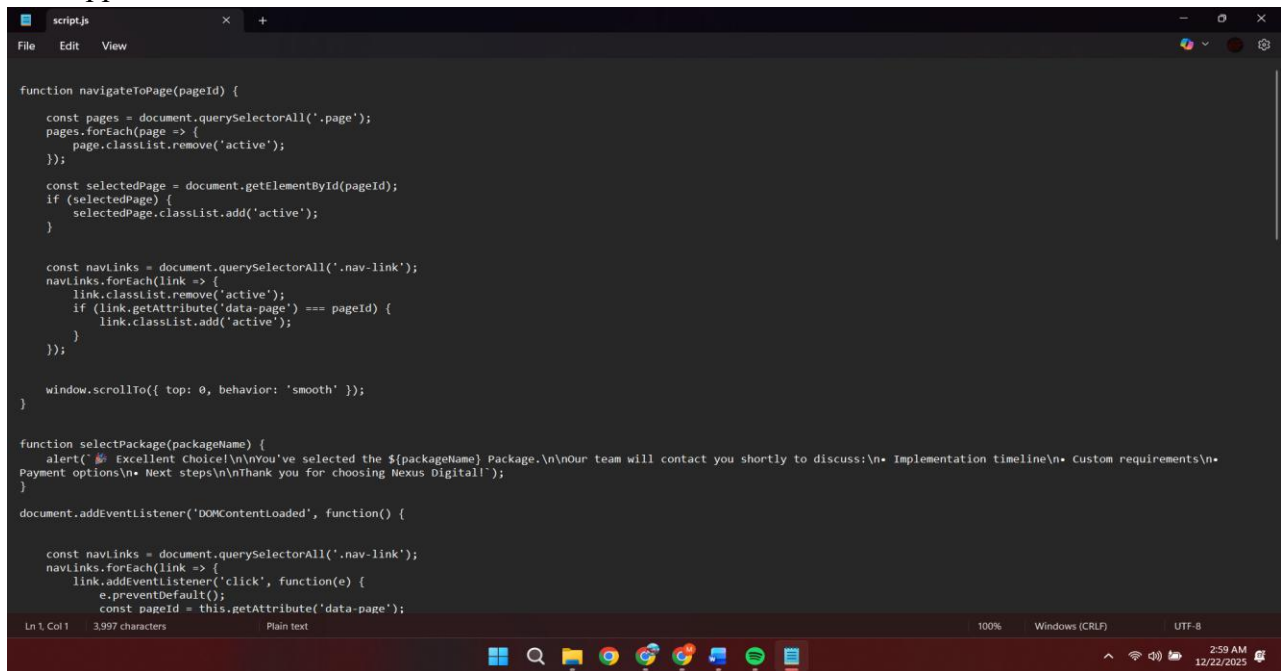
12. JavaScript Functionality and Logic

JavaScript is used for handling user interactions on the frontend, while the backend processes and stores user-submitted data.

The system uses API-based communication where:

- JavaScript captures form input
- Data is sent to backend endpoints using HTTP POST requests
- Backend validates received data
- Data is stored in Excel files for permanent record keeping
- Confirmation responses are sent back to the user

This approach simulates real-world client-server interaction.

A screenshot of a code editor window titled 'script.js'. The code defines two functions: 'navigateToPage' and 'selectPackage'. 'navigateToPage' handles removing the 'active' class from all pages, adding it to the selected page, and updating the navigation links. 'selectPackage' shows an alert message and adds an event listener to the navigation links. The code is in plain text, UTF-8 encoding, and the editor shows line 1, column 1 with 3,997 characters.

```
function navigateToPage(pageId) {  
  const pages = document.querySelectorAll('.page');  
  pages.forEach(page => {  
    page.classList.remove('active');  
  });  
  const selectedPage = document.getElementById(pageId);  
  if (selectedPage) {  
    selectedPage.classList.add('active');  
  }  
  
  const navLinks = document.querySelectorAll('.nav-link');  
  navLinks.forEach(link => {  
    link.classList.remove('active');  
    if (link.getAttribute('data-page') === pageId) {  
      link.classList.add('active');  
    }  
  });  
  
  window.scrollTo({ top: 0, behavior: 'smooth' });  
}  
  
function selectPackage(packageName) {  
  alert("Excellent Choice!\n\nYou've selected the ${packageName} Package.\n\nOur team will contact you shortly to discuss:\n• Implementation timeline\n• Custom requirements\n• Payment options\n• Next steps\n\nThank you for choosing Nexus Digital!");  
}  
  
document.addEventListener('DOMContentLoaded', function() {  
  
  const navLinks = document.querySelectorAll('.nav-link');  
  navLinks.forEach(link => {  
    link.addEventListener('click', function(e) {  
      e.preventDefault();  
      const pageId = this.getAttribute('data-page');  

```

12.1 Page Navigation Logic

To get the navigation working smoothly, we used JavaScript functions to handle everything on one page instead of loading a bunch of separate HTML files. Basically, JavaScript is the "brain" that decides what you see and what stays hidden.

Here's how it works:

- Hiding inactive sections: It ducks away the parts of the site you aren't using.
- Showing the right section: As soon as you click something, it brings that part front and center.
- Updating the links: It highlights the button you just clicked so you know exactly where you are.

This whole setup makes the site feel way faster and a lot more seamless since you aren't waiting for the screen to go white and reload every time you click a link.

12.2 Event Handling

We used JavaScript event listeners to make the site react whenever someone does something, like:

- Clicking navigation links to move around.
- Selecting service packages when they find something they like.

- Submitting forms once they've filled out their info.

By using the `addEventListener()` method, we kept the code super clean and organized, which makes it way easier to manage all those different user interactions without things getting messy.

12.3 DOM Manipulation

We use JavaScript to mess with the DOM (Document Object Model) in real time so the site feels alive. It basically lets us change things on the fly without a refresh:

- Swapping CSS classes: This is how we trigger those smooth animations or change how a button looks when it's clicked.
- Updating what you see: It swaps out the content in the main area instantly when you navigate.
- Highlighting the menu: It lets the site "remember" where you are by lighting up the right link in the sidebar.

Since these functions edit the DOM right as you're clicking, the transitions between sections feel super seamless and fast.

12.4 Interactive Home Page Reveal Effect

We added a cool interactive effect to the home page by mixing JavaScript with CSS. It's basically there to make the site feel more alive and fun to use.

Here's what it actually does:

- Tracks your mouse: It follows exactly where your cursor is moving on the screen.
- Updates the view on the fly: The visual area changes dynamically based on where you're looking or pointing.
- Boosts engagement: It just makes the landing page way more interesting to play around with.

This whole feature is a great example of how we can use event-based programming and some deeper DOM interaction to create something that isn't just a flat, boring page.

12.5 Service Package Selection Logic

Whenever someone picks a service package, we have JavaScript pop up a confirmation message right away. It's a pretty simple setup, but it does a few important things:

- Confirms the choice: It lets the user know the site actually registered their click.
- Feels more real: It mimics how a real-world transaction or interaction would feel.
- Keep people engaged: It adds a little bit of back-and-forth between the user and the site.

We used alert-based feedback to keep things straightforward and effective without overcomplicating the code.

12.6 Form Handling and Validation

Form handling is implemented using a combination of frontend JavaScript and backend APIs.

Key functionalities include:

- Input validation to ensure all required fields are completed
- Prevention of incomplete or invalid form submissions
- Backend verification of submitted data
- Automatic generation of unique request and complaint IDs
- Permanent storage of data in Excel files
- User confirmation messages after successful submission

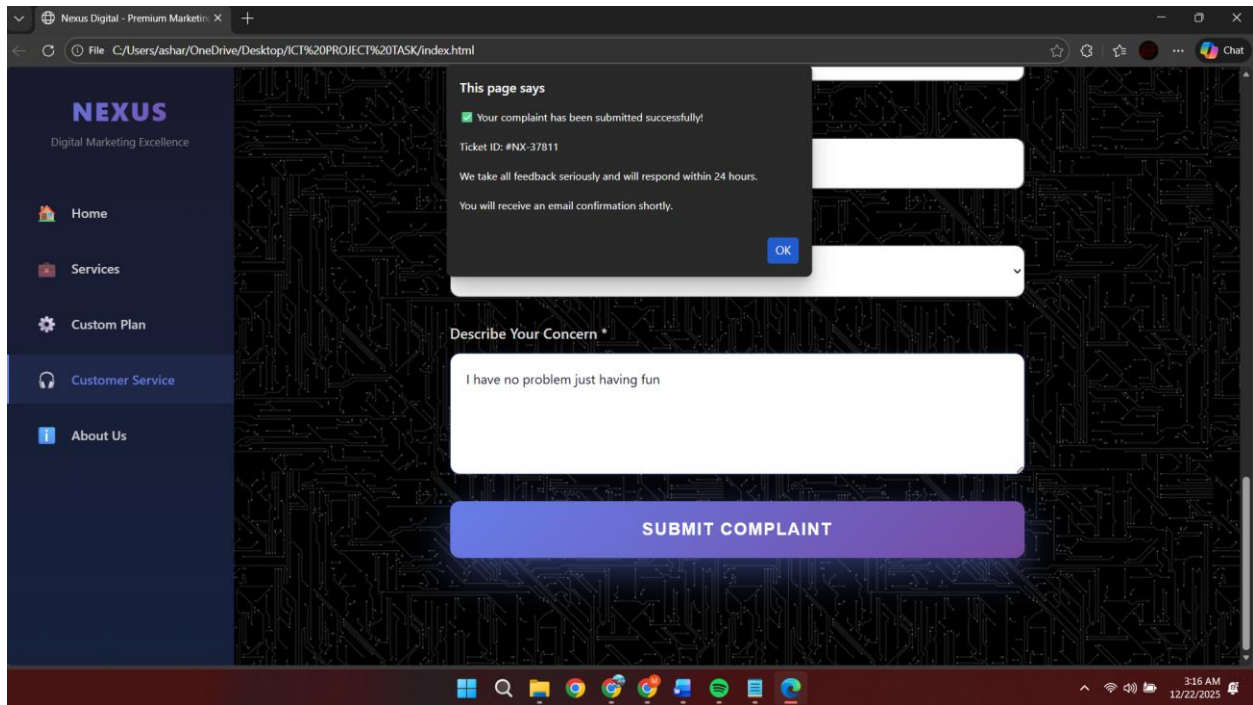
This process ensures data integrity and improves system reliability.

12.7 User Feedback and Alerts

We used JavaScript alerts to give people instant feedback whenever they do something on the site, like:

- Picking a service package: So, they know their choice was actually logged.
- Submitting a form: To confirm that their info is on its way.
- Sending a complaint: So, they aren't left wondering if we even got it.

Even though alerts are a pretty basic way to do things, they're super effective for letting users know exactly what the system is doing in response to their clicks.



12.8 Performance and User Experience Enhancements

JavaScript helps keep the site running fast by cutting out all the clunky stuff. Instead of making you wait for the browser to refresh every time you click, it just swaps content instantly.

Here's how it keeps things optimized:

- Skipping page reloads: It only updates the parts of the page that need to change, which saves a ton of time.
- Smoothing out the scroll: It makes moving between sections feel fluid rather than jumpy.
- Handling animations: It manages the heavy lifting for visuals so the site doesn't lag.

We also added a fade-in effect right when the page loads, which makes the whole entrance look way more polished and professional.

12.9 Error Prevention and Reliability

We put some basic error prevention in place to make sure the site doesn't crash or act weird while people are using it. It's mostly about making sure everything stays stable even if someone clicks something they shouldn't.

Here's what we focused on:

- Checking for page elements: Before the code tries to change anything, it double-checks that the element actually exists so it doesn't throw an error.

- Stopping empty forms: We blocked the "submit" button from working if the fields are totally empty, which saves us from getting blank entries.
- Handling random clicks: We set it up to handle unexpected user actions gracefully so the whole app doesn't just freeze up.

These checks are a big deal because they keep the site running smoothly and make sure the experience is consistent for everyone.

13. Testing and Validation

Testing was conducted manually to ensure correct system behavior.

13.1 Frontend Testing

- Navigation between sections was tested
- Responsive design was verified on different screen sizes
- Form input validation was tested

13.2 Backend Testing

- API endpoints were tested using form submissions
- Excel files were verified for correct data storage
- Validation checks for missing fields were tested
- Unique ID generation was confirmed

14. Limitations of the System

Even though our system does everything it needs to for our class assignment, there are still a few things it can't do yet. Since this was a semester project, we had to focus on the main features, which left us with some limitations:

- Storage: Right now, we're using Excel files to save data instead of a professional database like MongoDB or MySQL.
- Accounts: We didn't build a login system or any kind of user management, so anyone can access the forms without an account.
- Admin Access: There isn't an online dashboard for admins to log in and see the submitted data; you have to open the Excel file manually to see the entries.
- Hosting: The backend is currently set up to run locally on a computer rather than being hosted on a live cloud server.

For the scope of this end-of-semester project, we feel these trade-offs are okay, as the core functionality works exactly how we planned.

15. Future Enhancements

The system can be further improved through the following enhancements:

- Integration of a database for scalable data storage
- Development of an admin dashboard for data management
- Implementation of user login and authentication
- Deployment on a live cloud server
- Integration of email notifications

16. Conclusion

Overall, we're really happy with how this project turned out. We were able to build a full web-based system from the ground up using modern tools. By connecting a responsive frontend to a working backend, we created a site that handles forms, validates user input, and saves data just like a real application would.

Working on this together really helped us get a better handle on how client-server communication and APIs actually work in practice. It was a great way to see how data persistence works outside of just a theoretical lesson. In the end, our project hits all the requirements for the semester and gives a solid look at how real-world web apps are developed.